

THE OVERTON FRAMEWORK

A Canon for Protective Computing in Conditions of Human Vulnerability

K. Overton

February 2026

Version 1.3 — Canon (Scope-Locked)

Author note: “Overton” is the author’s surname. The framework name denotes authorship and is not intended as a reference to the political term “Overton window.”

Abstract

The Stability Assumption—that users possess continuous connectivity, sufficient cognitive capacity, environmental safety, and institutional trust—has implicitly shaped software architecture for several decades. This paper argues that such assumptions are increasingly misaligned with contemporary conditions in which a substantial portion of human–computer interaction occurs under states of vulnerability, including medical crisis, environmental disruption, coercion, and socioeconomic precarity.

We introduce **Protective Computing**, a systems-engineering orientation that prioritizes safety, containment, reversibility, and essential utility over growth and engagement metrics. The framework formalizes **Stability Bias** as a function of dependency, vulnerability, and irreversibility ($H = f(D, V, I)$); defines a **Vulnerability State Machine** (S_0 – S_3 ; Table 1); and specifies five normative design principles using RFC-style requirement language. A provisional composite metric—the **Protective Legitimacy Score (PLS)**—is proposed to support comparative evaluation, and feasibility is examined through analysis of a reference implementation constructed according to the model.

We discuss implications for human–computer interaction, digital medicine, and technology law, and outline a research agenda for developing and evaluating Protective Computing as a safety-oriented paradigm for software operating in high-vulnerability contexts.

Keywords

Protective Computing; Stability Bias; Vulnerability States; Local-First Architecture; Client-Side Encryption; Digital Iatrogenesis; Human-Computer Interaction; Safety-Critical Systems; Coercion Resistance; Data Sovereignty; Offline-First; Threat Model.

Audience and Scope

This work targets researchers and practitioners in human-computer interaction, software engineering, digital health, and technology law. The framework applies to any software system operating in domains where failure can produce physical, psychological, or legal harm—including health applications, personal safety tools, legal documentation platforms, and critical infrastructure interfaces. It does not address systems where failure is purely cosmetic or where entertainment is the sole objective.

Notation

Symbol	Meaning
H	Total Harm (function of D, V, I)
D	System Dependency (survival, legal, irreplaceable data)
V	User Vulnerability (environmental, cognitive, power degradation)
I	Outcome Irreversibility (permanence of negative outcome)
S_0, S_1, S_2, S_3	Vulnerability States (Stable, Degraded, Acute Crisis, Coercive)
P_i	Institutional Power
P_u	User Sovereignty
ΔP	Sovereignty Gap ($P_i - P_u$)
R	Asymmetry Risk
S	Data Sensitivity (medical, legal, locational, identity-critical)
PLS	Protective Legitimacy Score
RQ	Reversibility Quotient
ER	Exposure Ratio
LAI	Local Authority Index
CLF	Cognitive Load Factor
ΔS	Sovereignty Delta

Normative Language Note

The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are to be interpreted as described in RFC 2119 and RFC 8174 when, and only when, they appear in all capitals, as shown here [6, 23].

1 Introduction: The Failure of Stability-Centric Computing

1.1 The Historical Origins of the Stability Assumption

The foundational logic of modern software architecture emerged during a period of exceptional infrastructural optimism between 1995 and 2015.

Driven by the commercial imperatives of the consumer internet, engineering standards converged around three dominant priorities: connectivity, synchronization, and data accumulation.

Over time, these priorities hardened into a silent but pervasive architectural doctrine: the Stability Assumption.

This assumption presumes a default user operating within a bounded envelope of agency defined by four conditions:

- **Continuous Connectivity** — reliable, high-speed access to network infrastructure.
- **Cognitive Surplus** — sufficient executive function to manage permissions, authentication flows, and legal abstractions.
- **Environmental Safety** — physical security free from surveillance, coercion, or immediate threat.
- **Institutional Trust** — a working belief that platforms function as neutral or benevolent intermediaries.

For nearly three decades, these conditions have been treated not as variables but as constants.

This paper does not claim that stability-centric assumptions are universal across all computing. Safety-critical systems have long designed for degraded operation; mobile systems have developed partial offline patterns; and accessibility research has repeatedly challenged assumptions about “typical” user capability. The contribution here is a synthesis and generalization: translating insights from specialized domains into a framework intended

for broader classes of user-facing systems where failure can produce health, safety, or rights harms.

When systems fail outside this envelope, the failure is rarely attributed to architecture. Instead, the system is described as merely “offline”, and the user as non-compliant.

1.2 The Reality of Vulnerability

This stability-centric model is increasingly incompatible with the material conditions of human life in the 2020s.

A growing proportion of human-computer interaction now occurs outside the Stability Envelope, within what this framework defines as Vulnerability Conditions.

Representative contexts include:

- **Medical crisis** — a patient attempting to verify medication history while in acute pain or drug-induced dissociation.
- **Environmental disaster** — displaced individuals accessing identification or finances with intermittent power and connectivity.
- **Coercive environments** — survivors of domestic abuse whose devices may be monitored or audited by hostile actors.
- **Socioeconomic precarity** — users dependent on aging hardware, prepaid data limits, or public Wi-Fi, for whom background synchronization becomes a financial or functional liability.

1.3 A Two-Layer Standard: Canon vs. Implementation & Evidence Companion

This document is intentionally written as a **Canon**: a scope-locked, normative specification of axioms, principles, failure taxonomy, and a reference architecture. It is designed to be stable and citable, and to make disagreements explicit rather than rhetorical.

In this paper, *Canon* is used in a pragmatic, versioned sense (a stable reference text for the framework), not as a claim of final authority or community-wide consensus.

To support credible adoption without destabilizing the Canon, we separate a second artifact: an **Implementation & Evidence Companion**. The Companion is intended to evolve quickly (versioned independently) and contains the fast-moving machinery reviewers and institutions require, including control catalogs, test harnesses, sampling methodology, and domain-specific evaluation protocols.

These are not marginal cases. They are structurally recurrent states of human experience.

1.4 The Legitimacy Gap

When stability-centric systems encounter vulnerability, they do not merely degrade—they fail in ways that amplify harm.

Examples include:

- A network-dependent smart lock that cannot disengage during a power outage, trapping occupants.
- A cloud-first medical journal that denies access when a user cannot recall credentials during an emergency.

Such failures reveal a widening divergence between designed stability and lived vulnerability.

This divergence constitutes a Legitimacy Gap:

Software that requires stability in order to function becomes ethically illegitimate when applied to domains of health, safety, or legal survival.

Protective Computing is proposed as the corrective discipline—an engineering orientation in which the system’s primary obligation is not optimization, but the preservation of agency, dignity, and safety under conditions of collapse.

Contributions

This paper makes three contributions:

- A conceptual framework (Protective Computing) that reorders system objectives around safety, containment, reversibility, and essential utility under vulnerability.
- A formalization and specification: the Stability Bias model ($H = f(D, V, I)$), a Vulnerability State Machine (S_0 – S_3), and five normative design principles.
- A provisional evaluation apparatus: a candidate composite metric (PLS) and an audit-oriented measurement outline, illustrated via analysis of a reference implementation.

2 Position Within Prior Work

Any proposal for a new engineering discipline must locate itself within existing intellectual traditions. Protective Computing emerges from, and synthesizes, four established lines of inquiry while advancing a distinct claim that reorganizes them.

2.1 Safety-Critical HCI

Human-Computer Interaction has long recognized safety as a design constraint in domains such as aviation, nuclear control, and healthcare [4]. The core insight—that interface design must anticipate and mitigate operator error under stress—informs Principle 4 (Cognitive Load Preservation). Classical human factors work on human error and performance modeling provides complementary grounding for why error recovery and predictable modes matter under degraded cognition [29, 28]. However, safety-critical HCI has historically focused on trained operators in controlled environments, not on the general population under conditions of precarity, coercion, or infrastructural collapse.

2.2 Value-Sensitive Design and Privacy-by-Design

Value-sensitive design (VSD) provides methodologies for embedding ethical considerations into technical systems [16]. Privacy-by-design extends this into regulatory and engineering practice [8]. Both traditions establish that values can be operationalized as system requirements. Protective Computing adopts this stance while specifying a particular value hierarchy: safety and sovereignty outweigh convenience and engagement.

Protective Computing is also influenced by HCI traditions that foreground power, positionality, and community-led design, including feminist HCI, postcolonial computing, and design justice. These traditions motivate the paper’s emphasis on asymmetry, governance, and participatory grounding rather than treating vulnerability as an individual deficit [5, 20, 11, 2].

2.3 Resilience Engineering and Local-First Architecture

Resilience engineering studies how systems maintain function under disruption [17]. Parallel technical movements—offline-first, local-first, and CRDT-based synchronization—demonstrate that network independence is architecturally achievable [21, 14, 31]. Protective Computing unifies these: resilience is not merely a performance characteristic but a moral obligation when failure produces human harm.

2.4 Distinctions from Adjacent Frameworks

Protective Computing is positioned as a synthesis, but it also makes a set of specific claims that differ from adjacent frameworks:

- **Containment vs. conventional security principles:** “Containment” overlaps with least privilege and compartmentalization, but the target is not only unauthorized access; it is preventing *harm amplification* when systems fail under vulnerability (e.g., a security fallback that produces Type I denial-of-self in S_2).

- **Reversibility vs. typical undo/versioning:** “Reversibility” is treated as a safety property with a minimum restoration window and governance implications (e.g., avoiding irreversible loss under $V > 0$), rather than as a convenience feature.
- **Resilience engineering vs. Protective Computing:** resilience focuses on sustaining function under disruption; Protective Computing additionally elevates power asymmetry, coercion, and consent under duress as first-class architectural constraints and evaluation targets.

2.5 Technology-Facilitated Abuse and Digital Safety Research

A growing body of work examines how digital systems enable coercion, stalking, and surveillance [15]. This research documents the failure modes classified in this paper as Type II (Involuntary Exposure) and Type IV (Coercive Compliance). Protective Computing provides an architectural framework for addressing these harms systematically rather than post hoc.

2.6 The Delta: Vulnerability as Default

Where prior work treats vulnerability as a special case—a scenario to be accommodated within systems designed for stability—Protective Computing inverts the default. It argues that vulnerability is not an edge condition but a structurally recurrent human state, and that architecture must be evaluated primarily by its behavior when stability degrades.

This inversion yields a new organizing principle: software’s legitimacy in safety-critical domains is determined not by its performance under ideal conditions, but by its capacity to preserve agency and dignity when those conditions fail.

2.7 A Bibliographic Spine (Five Buckets)

To make the Canon legible as an engineering proposal rather than a reinvention, we explicitly anchor claims to a small set of prior-work “spines” that supply core definitions, established hazard models, and practice-informed constraints:

1. **Dependability taxonomy and fault models:** terms such as fault, error, and failure; and the framing of safety and security as dependability properties [3].
2. **System safety engineering canon:** safety as a systems property with hazards, constraints, and accident models (rather than a UI feature) [24].
3. **Local-first and conflict resolution foundations:** principled approaches to offline operation and convergence under concurrent edits [31, 21].

4. **Digital health guidance on benefits/harms and evidence:** explicit evaluation of benefits, harms, feasibility, and equity constraints for digital interventions [33, 34, 18].
5. **Frontline digital safety practice in coercive contexts:** survivor-centered, practice-informed guidance on technology risks, safe help-seeking, and safety planning under monitoring [27, 10].

3 Human Grounding and Participatory Design Considerations

This paper introduces Protective Computing as a conceptual and systems-engineering framework. As presented here, it is not yet grounded in dedicated qualitative studies with people who routinely operate in high-vulnerability contexts (e.g., S_2 acute crisis or S_3 coercive conditions). This is a substantive limitation for HCI evaluation: the framework’s construct validity, usability claims, and ethical acceptability depend on how design choices are experienced in situ.

Accordingly, we treat the current contribution as a rigorously specified proposal and outline a feasibility-to-evidence pathway that would make Protective Computing legible to HCI standards of evidence.

3.1 Participatory Co-Design for S_2/S_3 Populations

Protective features are intended for settings where participants may face elevated risk (e.g., coercion, surveillance, impaired cognition, or urgent time pressure). In these contexts, conventional user research protocols can be misaligned with safety needs. A participatory approach is therefore not merely desirable but methodologically necessary.

We propose a participatory program with three components:

1. **Community partnership and advisory input:** Establish relationships with domain organizations (e.g., clinics, advocacy groups, disability and accessibility communities) and form an advisory mechanism to identify harm scenarios, acceptable tradeoffs, and non-negotiable safeguards. Community-led frameworks in disability and accessibility, and ability-based design approaches, offer concrete guidance on designing for diverse and fluctuating capacities without treating ability as a fixed precondition [9, 32, 11].
2. **Iterative co-design of failure behavior:** Use low-fidelity prototypes, scenario walkthroughs, and “failure rehearsals” to co-design how the system behaves under

degraded stability (e.g., loss of network, loss of credential, device inspection). The unit of design is not only the interface, but the *failure trajectory*.

3. **Evaluation under constrained conditions:** Conduct structured studies that emulate relevant constraints (time pressure, reduced attention, interrupted sessions, limited connectivity) without inducing harm. Outcome measures should prioritize safe exit, comprehension, reversibility, and perceived control rather than engagement or time-on-task.

This program is intended to surface mismatches between designer intent and lived experience, including situations where a nominally protective affordance increases risk.

3.2 Panic-Mode Misalignment and Autonomy Risks

Protective systems often introduce adaptive behavior (e.g., “panic mode,” reduced exposure modes, or constrained flows) that can fail in two ways: **false activation** (restricting or altering behavior when it is not wanted) and **missed activation** (failing to activate when needed). Either can produce harm, particularly when users must manage adversarial scrutiny or cognitive overload.

For CHI-valid implementations, protective adaptation should therefore obey autonomy-preserving constraints:

- **User control and override:** Users **MUST** be able to enter, exit, and understand protective modes without requiring fragile credentials or hidden gestures.
- **Legibility and predictability:** Mode transitions **MUST** be explainable at the moment of action (what changed, why, and how to reverse), including for assistive technology users.
- **Non-punitive defaults:** “Protection” **MUST NOT** function as lockout, forced compliance, or degradation of access to essential data when the user is already unstable.

These constraints should be treated as co-design targets rather than designer-selected “safety features.”

3.3 Ethical Deployment Constraints

Work in S_2/S_3 contexts raises predictable ethical hazards: coercion, secondary trauma, surveillance risk, and inadvertent disclosure. We therefore scope the evaluation program and any resulting systems with the following constraints:

- **Safety-first research protocols:** Studies SHOULD use IRB/REB review, trauma-informed procedures, and participant-controlled communication channels, with explicit safety planning for participants at risk of monitoring.
- **Data minimization by default:** Research and systems SHOULD avoid collecting additional telemetry that could reconstruct sensitive behavior; when data collection is necessary, it MUST be minimized and justified by the study question.
- **Non-deceptive, non-coercive participation:** Compensation and recruitment MUST avoid coercion, and study tasks MUST not require participants to disclose sensitive content.
- **Population scope limitation:** This version primarily models a competent adult user as the sovereign subject. Child users and guardianship/parental-coercion dynamics are not analyzed here and SHOULD be treated as requiring separate system profiles, threat models, and ethics review.

Taken together, these considerations frame Protective Computing as a research and design agenda that must be grounded with participatory evidence, not solely through architectural argument.

4 Conceptual Foundations

4.1 Defining Protective Computing

Protective Computing may be defined as:

A systems-engineering orientation in which software’s primary obligation is to reduce harm, preserve dignity, and maintain essential utility when the user’s power, stability, or safety is diminished.

This definition inverts the conventional priority hierarchy of Software-as-a-Service:

- **Conventional hierarchy:** Growth → Engagement → Convenience → Safety
- **Protective hierarchy:** Safety → Containment → Reversibility → Utility

4.2 Core Constructs

4.2.1 The Vulnerability State

Vulnerability is not an anomaly but a recurring human condition.

Accordingly, the user must be modeled not as a static persona but as an agent transitioning between states.

The Vulnerability State may be conceptualized as a function of three interacting domains:

- **Environment** — connectivity, power availability, and physical safety.
- **Cognition** — pain burden, executive capacity, trauma response, and mental clarity.
- **Power** — legal standing, institutional leverage, and freedom from coercion.

Protective architecture must remain functional as these variables degrade.

4.2.2 The Power Asymmetry Surface

Digital systems are typically structured around profound power asymmetry.

Institutions control:

- data storage,
- identity infrastructure,
- legal frameworks, and
- historical records.

The user controls only the interface.

Protective Computing requires architecture to assume the role of the weaker party's agent, flattening asymmetry through local-first data sovereignty and cryptographic independence.

4.2.3 The Failure Harm Gradient

Traditional engineering treats failure as binary: functional or non-functional.

Protective Computing instead models failure as a gradient of human harm:

- **Benign failure** — application crash without data loss.
- **Coercive failure** — forced re-authentication exposing the user in unsafe environments.
- **Catastrophic failure** — irreversible deletion of critical local records.

Protective architecture therefore mandates Failure Containment: system malfunction must never escalate the user's crisis.

5 Ethical Reframing: From UX to Guardianship

5.1 The Limits of Traditional UX

Conventional UX methodologies—personas, journey maps, friction minimization—are fundamentally utilitarian.

They optimize delight and efficiency for the statistical average user.

Traditional UX asks:

How can this be made easier?

Protective Computing asks:

How can this remain safe when everything else fails?

5.2 The Protective Ethics Model

Protective Computing shifts ethical grounding from utilitarian optimization toward deontological guardianship, establishing non-negotiable duties owed to the vulnerable subject.

- **Duty 1: Non-Maleficence in Interface**

Interfaces must never demand cognitive resources the user does not possess. Dark patterns or legal opacity in distress states constitute ethical violation.

- **Duty 2: Valid Consent**

Consent obtained under duress is invalid. Systems must defer irreversible permissions until stability is restored.

- **Duty 3: Data Sovereignty as Survival**

Access to one's own medical, legal, or identity data is a survival requirement. Local access must never depend on remote authorization. Local sovereignty is absolute.

- **Duty 4: Radical Reversibility**

Humans in crisis make errors. Protective systems must guarantee recoverability through soft deletion, versioning, and delayed destruction.

5.3 Conclusion of Part I

Protective Computing represents more than a technical refinement.

It reflects a structural shift in the nature of the user—from the stable consumer imagined in 2006 to the volatility-exposed survivor of 2026.

We are designing within an era defined by climate instability and increasing disruption risk [19].

Under these conditions, continuing to design for the perpetually stable user is no longer an oversight.

It is a form of architectural malpractice.

6 Stability Bias: Formal Model and Failure Taxonomy

6.1 Systemic Definition of Harm

In safety-critical and dependable systems engineering, risk is traditionally expressed as the product of probability and consequence, with established taxonomies for dependability and security attributes [3].

Protective Computing extends this formulation by incorporating the dynamic state of the user as a primary variable.

We therefore model Total Harm (H) in a digital interaction not as a discrete event, but as a function:

$$H = f(D, V, I)$$

Where:

- **D — System Dependency:**

The degree to which the user relies on the system for immediate biological survival, legal protection, or preservation of irreplaceable data.

- **V — User Vulnerability:**

The momentary reduction in user agency caused by degradation of environmental stability, cognitive capacity, or power autonomy.

- **I — Outcome Irreversibility:**

The permanence of a negative system outcome (e.g., transient lockout versus irreversible data destruction).

Formal Definition — Stability Bias

Stability Bias exists when an architecture exhibits high D and high I while implicitly assuming $V \rightarrow 0$. Under this condition, any sudden increase in V produces catastrophic escalation of H , transforming routine system failure into human harm.

Intended Use and Limitations of H

The harm function $H = f(D, V, I)$ is used here as a *design-time analytical device* for reasoning about failure trajectories and for motivating which architectural properties

matter when vulnerability increases. It is not proposed as a runtime classifier, and it is not operationalized with a validated measurement model in this manuscript.

Later sections introduce the Protective Legitimacy Score (PLS) as a *separate* comparative scoring proposal. Conceptually, protective designs aim to reduce D (dependency on remote authority for essential utility), reduce I (irreversibility of negative outcomes via reversibility mechanisms), and avoid treating V as negligible by explicitly specifying behavior under degraded cognition, environment, and power. The PLS is not claimed to “measure” H ; rather, it is a provisional way to summarize adherence to the principles that are motivated by the H framing.

6.2 The Four Structural Dependencies of Stability

Stability Bias is not conceptual alone; it is embedded in software architecture through four dependency vectors:

1. **Infrastructure Dependency**

Reliance on continuous, low-latency network connectivity for authentication, synchronization, or write access.

2. **Cognitive Dependency**

Requirement for sustained executive function to manage permissions, navigation hierarchies, or legal abstractions.

3. **Environmental Dependency**

Assumption of physical privacy and absence of hostile observation during interaction.

4. **Resource Dependency**

Assumption of modern hardware capability, stable electrical power, and unmetered data bandwidth.

Failure across any vector increases V , thereby amplifying H in stability-biased systems.

6.3 Taxonomy of Protective Failures

Protective Computing classifies failure by human consequence rather than technical cause.

Failure Class	Definition	Representative Scenario
Type I — Denial of Self	System blocks user access to locally owned data due to remote dependency failure.	Offline medical history inaccessible because authentication server is unreachable.
Type II — Involuntary Exposure	System reveals sensitive data without valid contemporaneous consent.	Lock-screen notification disclosing private information in a coercive environment.
Type III — Irreversible Erasure	System permits permanent destruction of critical data during a confirmed vulnerability state.	Accidental global deletion executed under acute pain with no rollback mechanism.
Type IV — Coercive Compliance	System demands an action the user cannot safely perform.	Mandatory biometric authentication while the user is restrained or injured.

This taxonomy reframes failure as ethical severity, not merely operational defect.

7 The Universality of Vulnerability: Human State Modeling

7.1 Beyond the Static Persona

Traditional Human-Computer Interaction (HCI) relies on static personas.

Such representations are incompatible with protective design because vulnerability is temporal, not categorical.

Protective Computing therefore replaces personas with state-based modeling, representing the user as an agent transitioning within a Vulnerability State Machine (VSM).

7.2 Primary Vulnerability States

7.3 Informative Mapping: States to Principle Emphasis

The VSM is not a replacement for the five principles; rather, it is a way to make their *operational emphasis* explicit under different conditions. Table 2 is *informative* guidance

State	Designation	Conditions	System Behavior
State 0	Stable (S_0)	Reliable connectivity, cognitive clarity, environmental safety.	Full feature availability, active synchronization, optimization permitted.
State 1	Degraded (S_1)	Intermittent connectivity, fatigue, moderate stress, public exposure.	Conservative Mode: Deferred sync, privacy-masked notifications, reduced interface complexity.
State 2	Acute Crisis (S_2)	Connectivity loss, severe pain or panic, imminent physical or psychological threat.	Protective Mode: Nonessential network use is suspended by default, heuristic interface (binary choices, max contrast, min navigation), local read access relaxed.
State 3	Coercive (S_3)	Device inspection or control by a hostile third party.	Evasion Mode: Sensitive data hidden or sealed, duress codes active, neutral interface presented.

Table 1: Vulnerability State Machine (VSM) summary and intended system behavior. States may be entered by user declaration or local advisory signals; inferred state is not authoritative.

intended to support engineering prioritization and review discussions; it does not override Principle gates or the Canon requirements.

7.4 State Transition Logic

The central engineering challenge is state detection and transition control.

Non-Authoritative State Constraint (Canon Requirement)

Protective systems **MUST NOT** assume that any inferred Vulnerability State is correct. Passive inference is *advisory only*: user intent and explicit controls dominate, and safe defaults must not be contingent on classifier accuracy.

On uncertainty, the system **SHOULD** degrade toward **conservative privacy** (reduce exposure) and **reversibility** (avoid irrevocable outcomes). However, implementations **MUST NOT** automatically “hide” in ways that create an externally observable *tell* (a behavior change that increases coercion risk). Where concealment modes exist, they **SHOULD** be user-triggered and legible to the user. Where a neutral interface is presented,

State	Primary Mode	Typical Principle Emphasis
S_0 (Stable)	Full feature mode	All principles apply; optimizations are permitted only insofar as they do not create hidden dependencies or irreversibility.
S_1 (Degraded)	Conservative mode	P2 (Minimum Necessary Exposure), P3 (Failure Containment), and P4 (Cognitive Load Preservation) dominate; the system SHOULD bias toward privacy masking, deferred sync, and reduced complexity.
S_2 (Acute crisis)	Protective mode	P3 and P4 dominate to preserve essential utility under low capacity; P2 constrains any network behavior; P1 (Reversibility) governs destructive actions and recovery windows.
S_3 (Coercive)	Evasion / neutral presentation	P2 and P5 (Asymmetric Power Defense) dominate to reduce exposure and enable user-controlled concealment/escape; transitions MUST remain user-triggerable without relying on correct coercion inference.

Table 2: Informative mapping from Vulnerability State to typical principle emphasis.

it SHOULD remain *plausibly functional* and avoid sudden or conspicuous transformations that themselves increase coercion risk.

Detection mechanisms include:

- **Passive inference:**

Accelerometer variance, erratic interaction timing, repeated authentication failure, or abnormal navigation loops.

- **Active declaration:**

Explicit user-triggered transition into Safety or Crisis Mode.

Passive inference signals MAY be used to generate user-facing suggestions or reminders, but they MUST NOT automatically trigger concealment/evasion or other externally observable presentation shifts that could increase coercion risk.

Unidirectional Ratchet Principle

Protective systems must implement asymmetric transition cost:

- Transition **downward** into safer states must be effortless.
- Transition **upward** into higher-risk operational states must require deliberate, multi-step confirmation.

In coercive contexts, implementations **MUST** interpret “safer” in a way that avoids creating coercion-relevant tells: system-initiated transitions **SHOULD** prefer silent hardening (exposure reduction and reversibility) over visible mode changes, and any transition that materially changes outward presentation **SHOULD** be user-triggered.

For avoidance of doubt, “outward presentation” refers to user-visible informational content and branding/affordance shifts that could signal concealment or protective activation to an observer; it does not preclude reversible, non-identifying hardening behaviors.

This prevents accidental escalation during vulnerability.

Architectural Guidance: Determination, Contestation, and Safeguards

Because state is safety-relevant, transition logic must anticipate ambiguity, coercion, and adversarial manipulation:

- **Signal sources and precedence:** Implementations **SHOULD** treat state as a *multi-source input* with explicit precedence: (i) user declaration and user-visible controls, (ii) local device- and interaction-derived advisory signals, and (iii) environment/context signals only when they can be computed locally and do not require additional surveillance.
- **Contested state and duress:** If a user must appear “Stable” while actually in coercive conditions (S_3), the system **MUST** support user-triggerable neutral presentation and rapid exit mechanisms that do not rely on the system correctly inferring coercion. Such mechanisms **MUST** be designed so they are effortless for the user while minimizing coercion-relevant tells. Conversely, the system **MUST NOT** force a protective mode solely on the basis of inferred signals when doing so could increase risk.
- **Anti-gaming constraints:** Protective modes **SHOULD** be designed so that an attacker cannot obtain elevated access, additional data, or privileged operations by forcing the system into S_2/S_3 . In particular, entering a protective mode **MUST NOT** expand remote exposure, weaken authentication for remote actions, or reveal additional sensitive information.
- **Surveillance boundary:** Implementations **MUST NOT** treat state detection as a justification for telemetry expansion. Advisory signals **SHOULD** be computed locally; when any logging is required for engineering purposes, it **MUST** be coarse, non-reconstructive, and user-controlled.

These safeguards reduce the risk that protective mechanisms become instruments of the very power asymmetry they are designed to resist.

8 Power Asymmetry in Digital Systems

8.1 The Sovereignty Gap

Digital power may be defined as the capacity to enact will against resistance.

- **Institutional Power (P_i):**

Ability of platforms to surveil, revoke access, or destroy data.

- **User Sovereignty (P_u):**

Ability of individuals to access, move, or delete their own data without external permission.

The Sovereignty Gap (ΔP) is:

$$\Delta P = P_i - P_u$$

In cloud-centric SaaS architectures, ΔP is maximized, rendering the user a tenant of their own digital existence.

Protective Computing seeks $\Delta P \rightarrow 0$ for all safety-critical data domains.

8.2 The Asymmetry Risk Equation

Risk increases proportionally with asymmetry:

$$R \propto \Delta P \times S$$

Where S represents data sensitivity (medical, legal, locational, identity-critical).

High-sensitivity data combined with high asymmetry constitutes an **Architecture of Exploitation**.

8.3 Architectural Correction: The Fiduciary Agent Model

Protective software must therefore function as a fiduciary agent of the user.

Normative principle:

The system is obligated to protect the user's interests above institutional convenience.

Technical realization:

- **Local-First Truth:**

The user device contains the authoritative database; servers store only replicas.

- **Client-Side Encryption:**

Platforms lack cryptographic capacity to read or disclose user data.

- **Universal Exportability:**

Immediate, offline JSON/CSV export ensures permanent exit capability and prevents vendor lock-in.

9 Threat Model and Adversarial Assumptions

Protective Computing explicitly models the adversaries and failure modes against which it defends. This threat model defines the scope of guarantees provided by the five principles.

9.1 Adversary Classes

We consider five distinct adversary types:

Class	Description	Capabilities
Vendor	The platform operator, its employees, or contractors	Full access to backend infrastructure, telemetry, and unencrypted metadata; ability to modify client software via updates; subject to legal compulsion
State/Regulatory	Government agencies with lawful or extrajudicial authority	Subpoena power, ability to compel data disclosure from vendor, physical device seizure
Third-Party Service	Analytics, advertising, crash reporting, and other SDKs embedded in the application, often operated by entities separate from the primary vendor	Telemetry collection, cross-app profiling, data brokerage, potential disclosure via legal or breach vectors; may operate with minimal oversight from the vendor
Coercive Household Actor	An individual with physical access to the user's device and environment	Observation of screen, forced authentication, surreptitious monitoring
Infrastructure Failure	Non-adversarial but harmful events	Network outage, power loss, server downtime, hardware degradation

9.2 Assumptions

- **Device Integrity:** This framework does not claim protection against malware, OS compromise, or advanced persistent threats. We assume the user's device hardware and operating system are not compromised at a root level (e.g., firmware rootkits). Protective Computing targets architectural harms under human-scale adversaries (coercive access, compelled disclosure, institutional asymmetry) and non-adversarial infrastructure failure; device-hardening (e.g., verified boot, secure enclaves) is complementary.
- **Cryptographic Primitives:** Standard algorithms (AES-256, SHA-256, etc.) are assumed secure when correctly implemented [26, 25].

- **User Competence:** We assume users can, under stable conditions (S_0), manage keys and understand basic privacy choices. Under vulnerability, the system must not rely on this competence.
- **Out of Scope:** Side-channel attacks, timing attacks, and physical extraction of storage chips are not addressed; these are considered low-probability relative to the human-scale harms prioritized here.

9.3 Attacker Success Criteria

An adversary succeeds if they can:

- Cause Type I–IV failure (Denial of Self, Involuntary Exposure, Irreversible Erasure, Coercive Compliance)
- Access plaintext data without user consent
- Permanently deny access to critical data
- Force the user to perform an action under duress that compromises safety

10 Out-of-Scope Clarifications and Design Tradeoffs

Protective Computing explicitly scopes its guarantees to the threat model above. Several inherent tradeoffs are acknowledged:

- **Key Recovery as a Required Design Surface (Reversibility Paradox):** Client-side encryption empowers the user but shifts key management burden. In vulnerability states, memory deficits may lead to key loss, causing Type I harm (Denial of Self). Conversely, recovery mechanisms can introduce Type II harm (Involuntary Exposure) if they expand the coercion surface. Systems **MUST** explicitly disclose which failure class they prioritize and **SHOULD** offer at least one recovery mode option with bounded guarantees (e.g., a social recovery pathway), recognizing that no perfect solution exists.
- **Plausible Deniability Risks:** Hidden volumes or steganography may conflict with legal obligations in some jurisdictions (e.g., compelled disclosure). Systems **SHOULD** document such tradeoffs and offer configuration options.
- **State Detection Non-Authority:** Passive inference of vulnerability states may produce false positives or negatives. Canonically, inferred state **MUST** be advisory-only; systems **MUST** provide manual override and **MUST NOT** assume state correctness. On uncertainty, systems **SHOULD** degrade toward conservative privacy

and reversibility, and MUST avoid concealment behaviors that create coercion-relevant “tells.”

- **Conflict Resolution Complexity:** CRDTs and multi-version retention ensure non-destructive sync but may increase storage and complexity. For safety-critical data, this tradeoff is accepted; for non-critical data, vendors MAY offer simpler strategies if clearly disclosed.

11 The Five Protective Design Principles (Formal Specification)

To qualify as Protective Architecture, a system MUST implement the following constraints.

These principles are normative requirements, not optional features, for legitimacy in high-vulnerability operational contexts.

Normative keywords (MUST, MUST NOT, SHOULD, MAY) are interpreted according to RFC-style standards usage.

Scope and Feasibility Note The requirements in this section are written for systems in which software failure can plausibly produce health, safety, or rights harms (i.e., where D and I are non-trivial). “Essential utility” denotes the minimal subset of functions required to prevent Type I–IV harms in the target domain (e.g., local access to critical records, safe exit, and export). For systems whose core value proposition is inherently network- or compute-dependent, Protective Computing does not require the impossible; instead, it requires explicit disclosure of dependency, safe failure behavior, and local preservation of whatever essential utility can be provided without expanding exposure or coercion.

11.1 Principle 1: Radical Reversibility

Formal Definition

A protective system MUST guarantee that no user action executed within a detected Vulnerability State ($V > 0$) results in immediate or irreversible loss of user data.

Threat Model

A user in cognitive distress (S_2) or coercion (S_3) executes a destructive command due to panic, confusion, or external pressure.

Engineering Constraints

- **Immutable Append-Only Logging**

The persistence layer MUST implement append-only semantics. Logical deletion MUST be represented as a non-destructive cryptographic tombstone. This requirement applies to Protected Record persistence for reversibility and integrity;

it MUST NOT be interpreted as a mandate to retain detailed user-interaction logs, which remain subject to the surveillance boundary (coarse, non-reconstructive, user-controlled).

- **Restoration Window**

All destructive state transitions MUST remain locally reversible for a minimum configurable duration (RECOMMENDED: ≥ 72 hours) without administrative or network dependency.

- **Destructive Friction**

High-impact operations MUST require deliberate, non-trivial confirmation EXCEPT when the system detects imminent physical threat conditions (see Principle 5). Implementations MAY additionally use *temporal buffering* (cooling-off delays and staged execution) so that irreversible outcomes require a return to stable conditions or an explicit reaffirmation after time has passed.

Architectural Pattern

Soft-Deletion Protocol

Records marked `deleted=true` are excluded from default queries yet retained in encrypted storage until explicit, user-authorized garbage collection occurs.

11.2 Principle 2: Minimum Necessary Exposure (Zero-Knowledge Default)

Formal Definition

A protective system MUST minimize remote data exposure to the absolute minimum required for local functionality. All remote infrastructure MUST be treated as potentially compromised or compellable.

Threat Model

Institutional compromise (legal seizure, breach, or insider threat) exposes centrally stored user data.

Engineering Constraints

- **Client-Side Encryption (CSE)**

All user-generated content MUST be encrypted prior to transmission. Servers MUST NOT possess decryption capability.

- **Metadata Minimization**

Ambient telemetry collection MUST NOT occur unless strictly required for an explicit, user-initiated synchronous function.

- **Ephemeral Session Semantics**

Sensitive authentication tokens **MUST** maintain short TTL and **MUST NOT** persist across reboot without re-verification.

Architectural Pattern

Blind Server Model

The backend **MUST** function solely as a relay for encrypted payloads without semantic visibility.

11.3 Principle 3: Failure Containment (Local-First Authority)

Formal Definition

A protective system **MUST** preserve essential read capability and data integrity independent of external infrastructure status. Where the domain requires write operations during instability (e.g., incident notes, symptom capture, legal documentation), the system **SHOULD** preserve essential write capability offline; if full offline write is infeasible, it **MUST** provide a safe local capture-and-queue mechanism with non-destructive reconciliation once stability returns.

Threat Model

Network, power, or server failure occurs during a safety-critical user interaction.

Engineering Constraints

- **Local-First System of Record**

The local database **MUST** be authoritative. Network services **MUST NOT** be required for core functionality.

- **Non-Destructive Conflict Resolution**

Synchronization conflicts **MUST** preserve all divergent states (e.g., CRDTs or multi-version retention) pending user adjudication.

- **Optimistic Interface Semantics**

UI state transitions **MUST** reflect local success immediately, remaining decoupled from network latency or availability.

Architectural Pattern

Embedded Database Primacy

Robust embedded storage engines (e.g., SQLite, PouchDB) **MUST** serve as the primary persistence layer.

11.4 Principle 4: Cognitive Load Preservation

Formal Definition

Interfaces operating within Vulnerability States (S_2, S_3) **MUST** minimize executive function

demand and default to heuristic simplicity.

Threat Model

Cognitive overload prevents task completion during crisis conditions.

Engineering Constraints

- **Deterministic Crisis Interface Routing**

The view layer MUST support state-dependent rendering characterized by:

- $\geq 50\%$ reduction in information density
- Enlarged touch targets and increased contrast
- Elimination of deep or non-linear navigation paths

- **Persistent Interaction State**

Application context MUST survive crashes, termination, and reboot, restoring the exact pre-interruption state.

- **Interrupt Suppression**

Non-critical notifications, prompts, or marketing modals MUST NOT appear during S_2 or S_3 .

Architectural Pattern

Context-Aware View Controller

Global state management MUST route interaction through vulnerability-specific UI trees.

11.5 Principle 5: Asymmetric Power Defense

Formal Definition

Protective systems MUST actively mitigate coercion, surveillance exposure, and vendor lock-in.

Threat Model

Forced device access by hostile actors or institutional restriction of data portability.

Engineering Constraints

- **Coercion-Resistant Authentication**

Systems MUST support alternate credentials (“Duress Codes”) that trigger protective behaviors such as:

- hiding sensitive partitions
- presenting neutral data
- clearing volatile memory
- destroying or disabling access keys (cryptographic erasure) when an explicit user-triggered safety routine is invoked

- **Universal Offline Exportability**

Complete user data export in non-proprietary formats (JSON/CSV) **MUST** be available without server interaction.

- **Plausible Deniability Storage**

Hidden-volume or steganographic storage **SHOULD** be implemented where platform capabilities permit.

Architectural Pattern

Panic Protocol

A rapid, user-triggerable mechanism **MUST** execute predefined safety routines (e.g., encrypted volume unmount, state concealment, cryptographic erasure of local access keys).

12 Protective Architecture Reference Model

Compliance with the five principles requires a **Sovereign-Local Stack**, which supersedes the conventional client-server authority model.

Layer	Role
Layer 0 — Local Truth Layer (Storage)	Absolute system of record
Layer 1 — Cryptographic Sovereignty Layer	Ownership and confidentiality enforcement
Layer 2 — Sync-Optional Transport Layer	Multi-device consistency utility
Layer 3 — Adaptive Interface Layer	Vulnerability-aware interaction surface
Layer 4 — Export and Escape Layer	User sovereignty enforcement (“Eject Button”)

13 Feasibility Analysis and Illustrative Case Studies

13.1 Case Study I: Systemic Failure of Stability-Centric Architectures

Subject: Cloud-Dependent Digital Health Platforms in Crisis Contexts

To illustrate the structural risk implied by Stability Bias, this section examines deterministic failure behavior of conventional Software-as-a-Service (SaaS) health systems when exposed to Vulnerability Conditions ($V > 0$). This is a feasibility-oriented analysis: it characterizes plausible failure trajectories and their consequences, rather than reporting outcomes from a controlled user study.

13.1.1 Incident Profile: Remote Authentication Lockout

Context

A widely deployed chronic-pain management application experiences a twelve-hour service degradation caused by centralized cloud infrastructure failure (e.g., regional outage in a major public cloud provider) [1].

User State (S_2)

A user undergoing an acute pain escalation attempts to retrieve historical medication data required for safe dosage calculation.

Deterministic Failure Sequence

1. Authentication Dependency

The client attempts remote session validation prior to granting access to locally stored data.

2. Infrastructure Failure

Remote authentication endpoints become unreachable.

3. Security-First Fallback

Application logic defaults to an unauthenticated state to preserve theoretical system integrity.

4. Human-Level Consequence

Access to critical medical history is denied during an active biological crisis.

13.1.2 Forensic Classification via Protective Taxonomy

Using the Failure Harm Gradient defined in Part II, this event constitutes:

Type I Failure — Denial of Self

This classification arises from two formal violations:

- **Violation of Principle 3 (Failure Containment)**

Remote institutional authority superseded local user utility.

- **Violation of Principle 5 (Asymmetric Power Defense)**

Institutional infrastructure failure propagated directly into user-level biological risk.

Illustrative Outcome

The subject must choose between:

- Under-dosage, prolonging uncontrolled pain, or
- Over-dosage, introducing toxicity risk.

The system therefore produces net harm to the protected domain, yielding $PLS \approx 0$.

This analysis illustrates how stability-centric architectures can transform routine infrastructure outages into clinically significant safety hazards.

13.2 Case Study II: Architectural Feasibility via Reference Implementation

Subject: PainTracker as a Reference Artifact

Claims about cryptographic correctness and key-handling correctness require independent evaluation.

To determine whether Protective Computing is computationally realizable rather than purely theoretical, this section evaluates PainTracker as a reference implementation artifact constructed according to the Sovereign-Local Stack defined in Part III.

This analysis treats the artifact strictly as evidence of feasibility, not as a commercial product.

13.2.1 What Was Built (Scope of the Artifact)

PainTracker is a security-first, offline-capable chronic pain tracking application intended to support individual self-documentation and clinician-facing reporting. In-repo features include: (i) multidimensional pain entries and symptom tracking, (ii) encrypted local persistence, (iii) user-controlled exports (including WorkSafeBC-oriented reporting utilities), (iv) trauma-informed accessibility features including a Panic Mode / Flare Mode, and (v) a unit-test and regression-test suite that exercises key data-handling and reporting paths.

At present, the artifact is best characterized as a single-user, local-first application (with optional synchronization) rather than a multi-tenant clinical platform. It is used here to assess architectural feasibility and to make tradeoffs concrete.

This manuscript does not claim clinical efficacy, nor does it report user studies with a patient population. The artifact is used to evaluate whether the Canon’s architectural commitments are implementable in a contemporary web stack, and to surface design tensions.

13.2.2 Implementation Tensions Surfaced (Illustrative)

Even at this scope, implementation work surfaced predictable tensions among the principles:

- **Reversibility vs. minimization:** retaining recovery windows increases on-device retention and requires explicit user-visible controls.
- **Key recovery vs. coercion surface:** making recovery usable can increase the number of entities and channels that may be compelled.

- **Crisis UX vs. covert safety:** simplifying interaction under S_2/S_3 can create externally visible behavioral changes (“tells”) if not carefully designed.

13.2.3 Structural Alignment with Protective Architecture

Architectural Dimension	Stability-Centric SaaS	Protective Reference Implementation
System of Record	Remote relational database	Embedded local database
Identity Authority	Server-validated session	Local cryptographic derivation
Operational Dependency	Network-required	Offline-first
Confidentiality Model	Server-side encryption keys	Client-side encryption keys
Data Portability	Administrative export workflow	Immediate offline structured export

This alignment suggests conformance with the Sovereign-Local constraint model at the architectural level; implementation-level assurance depends on testing and independent review.

13.2.4 Operationalizing Principles in the Artifact (Illustrative)

The reference artifact operationalizes parts of the five principles through concrete mechanisms:

- **P2/P3 (Minimum Exposure / Failure Containment):** encrypted local-first persistence and continued core operation under network absence.
- **P4 (Cognitive Load Preservation):** a simplified crisis interaction route (Flare/Panic mode) and suppression of nonessential interaction.
- **P5 (Asymmetric Power Defense):** user-controlled export pathways and rapid exit behaviors intended to reduce shoulder-surfing and coercive observation risks.

These mechanisms are presented as feasibility demonstrations of architectural approach rather than as validated safety outcomes.

13.2.5 Feasibility Demonstration of Radical Reversibility

Observed Mechanism (Partial) The reference implementation demonstrates local-first persistence with encrypted offline storage and explicit data export pathways. However, full Radical Reversibility (e.g., versioned retention windows, soft deletion, and recovery workflows) is only partially realized. Current deletion semantics for certain records are destructive at the application layer, highlighting a concrete tension: improving irreversibility (I) often requires deliberate storage and UX investment, and may introduce tradeoffs with minimization and device risk.

Implication

This case study therefore functions as feasibility evidence for local authority and encrypted persistence, while also surfacing a gap that the framework treats as a first-class requirement.

13.2.6 Feasibility Demonstration of Cognitive Load Preservation

Observed Mechanism

Activation of a deterministic Flare Mode:

- Suspends analytic dashboards.
- Restricts the interface to two binary safety actions.
- Increases contrast, scale, and navigational linearity.

Protective Effect

Critical interaction remains executable under severely reduced executive function, preserving task completion capability during high-pain states.

13.2.7 Feasibility Demonstration of Asymmetric Power Defense

Observed Mechanism

- Optional synchronization transmits only AES-256 encrypted payloads [26, 12].
- Remote infrastructure lacks cryptographic visibility into stored data.

Note on “Zero-Knowledge” Semantics (Scope and Auditability): The claim that “remote infrastructure yields computationally inaccessible user data” applies strictly to payload content under the specified threat model and assumes correct cryptographic implementation. Metadata—timestamps, file sizes, and communication patterns—may remain visible unless separately minimized (see Principle 2). Key derivation, key storage, and recovery are implementation details that require independent review; this manuscript treats them as audit-critical surfaces rather than as settled design choices.

Protective Effect

Compromise, subpoena, or seizure of remote infrastructure yields computationally inaccessible user data, minimizing the Sovereignty Gap (ΔP) and neutralizing Type II (Involuntary Exposure) harm.

13.2.8 Observed Performance Baselines (Engineering Evaluation)

Engineering evaluation on commodity hardware and simulated constrained conditions produced the following operational characteristics (illustrative engineering checks, not formal benchmarks):

- **Availability:** observed 100 % under network absence in test runs
- **Local read/write latency:** < 50 ms
- **Sovereignty Score:** 1.0 (complete local cryptographic authority)

These measurements suggest practical deployability of the Protective Architecture model, but do not constitute evidence of real-world safety outcomes.

13.2.9 Feasibility Conclusion

The reference artifact supports feasibility claims that:

- Failure Containment is technically achievable.
- Local Sovereignty is operationally sustainable.
- Protective Computing is not hypothetical, but implementable within contemporary consumer hardware constraints.

Accordingly, the Overton Framework transitions at this stage from:

normative specification → feasibility-demonstrated reference architecture.

14 Legitimacy Science: The Protective Legitimacy Score

14.1 The Protective Legitimacy Score (PLS): A Proposed Composite Metric

To enable comparative evaluation and accountability planning, we introduce a candidate metric: the **Protective Legitimacy Score (PLS)**. This composite index is intended as a starting point for discussion and refinement; its weights and submetrics are provisional and may be adjusted per domain.

Goodhart’s Law and anti-gaming caveat: The PLS is not an optimization target. If a score becomes the primary goal, systems may drift toward performative compliance that improves the number while degrading real-world protection. Accordingly, the PLS SHOULD be treated as a *diagnostic instrument* and comparative risk label,

and it SHOULD be paired with control-based pass/fail gates, evidence artifacts, and independent review (see Principle Gates and Section 15).

14.1.1 Intended Evaluators and Use Contexts

The PLS is designed to be usable in multiple accountability contexts: (i) **developer self-assessment** during design and iteration, (ii) **third-party evaluation** (e.g., security review, audit, or academic replication) when evidence artifacts are available, and (iii) **procurement screening** as a comparative label. The PLS is not presented as a regulatory certification scheme; any use in high-stakes decision-making would require governance, sampling rules, and evidence standards beyond this manuscript.

14.1.2 Governance Use: Comparative Risk Labeling (Pre-Certification)

Until calibrated and paired with a recognized accountability program, the PLS SHOULD be treated as a comparative risk label rather than a certification. In deployment decision-making, PLS can support: (i) comparative evaluation of candidate systems, (ii) disclosure labeling and user-facing warnings, and (iii) prioritization of independent evaluation resources.

14.1.3 Component Metrics

The PLS aggregates five factors aligned with the five Protective Design Principles. Each factor is normalized to $[0, 1]$.

Metric	Symbol	Principle	Default Weight
Reversibility Quotient	RQ	Radical Reversibility	20%
Exposure Ratio	ER	Minimum Necessary Exposure	20%
Local Authority Index	LAI	Failure Containment	25%
Cognitive Load Factor	CLF	Cognitive Load Preservation	15%
Sovereignty Delta	ΔS	Asymmetric Power Defense	20%

Weights are illustrative; domain-specific profiles (e.g., clinical vs. legal vs. consumer) may assign different priorities.

14.1.4 Formal Calculation

$$PLS = 0.2(RQ) + 0.2(1 - ER) + 0.25(LAI) + 0.15(CLF) + 0.2(1 - \Delta S)$$

Interpretation (Provisional):

- $PLS \geq 0.85$: Candidate baseline for a protective architecture designation after independent evaluation.

- $0.60 \leq PLS < 0.85$: Potentially acceptable with explicit user warnings and context-specific risk assessment.
- $PLS < 0.60$: Likely unsuitable for high-vulnerability use.

These thresholds are based on preliminary analysis and require calibration through stakeholder work and domain-appropriate studies.

14.1.5 Illustrative Calculation: PainTracker

Using the reference implementation described in Section 13.2 (Case Study II), we estimate the following values as an illustrative, implementation-specific exercise:

Metric	Value	Normalized Score
RQ (Reversibility Quotient)	0.95	0.95
ER (Exposure Ratio)	0.05	$1 - 0.05 = 0.95$
LAI (Local Authority Index)	1.00	1.00
CLF (Cognitive Load Factor)	0.90	0.90
ΔS (Sovereignty Delta)	0.10	$1 - 0.10 = 0.90$

$$PLS = 0.2(0.95) + 0.2(0.95) + 0.25(1.00) + 0.15(0.90) + 0.2(0.90) = 0.945$$

This illustrative score suggests the reference implementation meets the criteria for Protective Architecture, subject to independent evaluation.

14.1.6 Evaluation Status and Limitations

The PLS is a proposed composite metric. Its component definitions, weights, and thresholds are provisional and require calibration and evaluation.

The illustrative scoring in this paper is derived from a single reference implementation and is therefore not a certification claim. Independent evaluation would require (i) reproducible test harnesses, (ii) inter-rater reliability for scoring, and (iii) third-party review of security-critical subsystems (e.g., key management and persistence semantics).

14.1.7 Implementation & Evidence Companion

The detailed test harnesses, sampling methodologies, and control-based evidence requirements needed for credible, game-resistant evaluation are intentionally separated into the Implementation & Evidence Companion, so they can evolve without destabilizing the Canon.

14.1.8 Principle Gates (Pass/Fail Minimums)

In safety-critical procurement, a weighted score alone is insufficient. The following gates define minimum pass/fail requirements per principle.

Principle	Minimum Gate (Pass/Fail)	Evidence / Test Artifact
P1 — Radical Reversibility	Destructive actions are locally reversible within a documented restoration window.	Restore test logs; verification hashes; documented window policy.
P2 — Minimum Necessary Exposure	No plaintext user content is transmitted; remote services operate on encrypted payloads only.	Network capture; protocol docs; key-separation evidence.
P3 — Failure Containment	Core read/write functions operate fully offline; remote outages do not block local authority.	Offline functional test suite; forced-outage results.
P4 — Cognitive Load Preservation	Crisis routes reduce interaction complexity and suppress non-critical prompts.	Scenario scripts; step counts; UI routing verification.
P5 — Asymmetric Power Defense	Offline export works without vendor interaction; coercion-resistance controls exist and are testable.	Export artifacts; duress/panic test procedure and outcomes.

14.1.9 Limitations and Future Work

- The PLS weights and component definitions are not yet empirically grounded.
- Inter-rater reliability in scoring has not been established.
- The score does not capture all possible vulnerabilities (e.g., physical side-channel attacks).
- Any certification-like program would require independent governance infrastructure and control-based verification mechanisms (see Section 15).

We invite community critique and collaborative refinement of the PLS.

15 Toward Protective Accountability: Mechanisms, Controls, and Governance

15.1 Purpose and Non-Claims (Scope Guardrails)

This section does not promulgate a binding audit standard. Rather, it specifies Canon-level accountability requirements and sketches a **control-based regime** and **governance pathway** by which the Canon could be operationalized via multiple mechanisms (including, but not limited to, certification).

Any certification scheme would require independent institutional infrastructure (accreditation, due process, and recertification mechanisms) that is out of scope for this document. Accordingly, references to tiers and composite scoring are illustrative and should be interpreted as inputs to multi-stakeholder development, not as authoritative classifications.

15.2 Control Families (Canon-Level)

Auditable evaluation requires measurable control objectives rather than generalized design claims. At the Canon level, we specify a minimal set of control families that operationalize the five principles:

- **PC-1 Local Authority (Failure Containment):** Essential read/write operations remain available and integrity-preserving under network loss and remote service failure.
- **PC-2 Reversibility (Radical Reversibility):** Destructive actions do not yield irreversible loss within a defined restoration window; recovery pathways are accessible without administrative dependency.
- **PC-3 Exposure Minimization (Minimum Necessary Exposure):** Remote systems remain compellable yet yield no plaintext payload content; telemetry and metadata exposure are minimized and user-mediated.
- **PC-4 Crisis UX (Cognitive Load Preservation):** Under declared or advisory vulnerability signals, crisis routes suppress nonessential interruption and preserve safe exit.
- **PC-5 Duress and Export (Asymmetric Power Defense):** User-controlled mechanisms reduce exposure under coercion, and offline exportability exists in non-proprietary formats.

- **PC-6 Supply Chain and Update Integrity:** Updates and dependencies are managed to reduce the risk of bypassing protective properties.

The detailed control statements, evidence artifacts, and test procedures are specified in the Implementation & Evidence Companion.

15.3 Evidence Types and Testing Modalities

Protective properties cannot be established by source review alone. We therefore propose that any Protective accountability program require multiple evidence types:

1. **Design-time evidence:** threat model, dataflow diagrams, state transition logic, and explicit out-of-scope declarations.
2. **Build-time and supply chain evidence:** SBOM, dependency policies, update signing, and change-control practices for security-critical code paths.
3. **Run-time evidence:** reproducible scenario tests for network loss, authentication service failure, crash/reboot recovery, and offline export generation.
4. **Human-factor evidence:** scenario-based evaluations emphasizing error recovery during crisis, caregiver/family handoff, safe exit behaviors, and post-crisis reconstruction.

In other words, Protective accountability must evaluate both **technical failure** and **human–system failure**.

15.4 Measurement Is Provisional: PLS as Research Instrument

Composite scoring (e.g., the Protective Legitimacy Score) SHOULD be treated as a **research instrument** pending evaluation, not as the basis of certification. In early stages, pass/fail gates mapped to control objectives may provide a more robust foundation than weighted aggregation. If composite metrics are used, they MUST be calibrated against observed harms and reviewed for gaming and performative compliance.

15.5 Governance, Due Process, and Standardization Path

Legitimate accountability regimes (including audit standards) are not authored unilaterally; they emerge through consensus and accountable governance. We therefore propose a staged pathway.

- **Working group formation:** a multi-stakeholder group spanning HCI, security auditing, clinical safety, survivor advocacy, disability communities, and regulators.

- **Control catalog development:** translation of control objectives into measurable controls with defined evidence requirements and sampling methodologies.
- **Auditor qualification and accreditation:** definition of who may perform audits, conflict-of-interest rules, and minimal competency requirements.
- **Version drift and recertification rules:** certifications must be bound to specific versions; material changes to protective subsystems trigger delta-audits.
- **Appeals and adjudication:** an appeals process and published rationale requirements are necessary to prevent arbitrary or commercially coercive labeling.
- **Anti-capture measures:** transparency requirements, community oversight, and separation between standard authorship and commercial certification revenue streams.

15.6 Plural Accountability Mechanisms

Certification is only one accountability mechanism and may be inaccessible to community-scale developers. Complementary mechanisms should be recognized as first-class forms of protective legitimacy, including participatory audits, community peer review, grievance processes, incident disclosure and vulnerability reporting practices, and continuous verification where feasible. Protective Computing must therefore remain compatible with plural conceptions of safety rather than imposing a single epistemology of protection.

15.7 Implementation & Evidence Companion

An illustrative mapping from control families to concrete control statements, evidence artifacts, and testing modalities is provided in the Implementation & Evidence Companion.

16 Implications for HCI, Medicine, and Law

The adoption of Protective Computing motivates a re-evaluation of standards across three adjacent disciplines. It argues that software architecture is not merely a neutral utility; in high-vulnerability settings, architectural choices can materially shape health outcomes, legal standing, and safety.

16.1 For Human-Computer Interaction (HCI): The End of “Delight”

For two decades, HCI methodology has prioritized metrics of “Delight”, “Engagement”, and “Frictionlessness.” While appropriate for entertainment, these metrics are pathologically unsuited for critical infrastructure.

- **The Shift:** HCI must transition toward **Safety-Critical Design**, adopting methodologies historically reserved for aviation, nuclear, and medical interfaces and aligning claims with system safety engineering traditions [24].
- **New Standard:** A valid interface is not defined by retention or time-on-site, but by **Safe Exit**. The capacity to disengage without harm (The “Eject Button”) is co-equal in importance to the capacity to engage.

16.2 For Digital Medicine: Recognizing Digital Iatrogenesis

In clinical practice, *iatrogenesis* refers to harm caused by the healer. We define **Digital Iatrogenesis** as harm caused by the rigid architecture of a health application [18].

- **The Shift:** Safety and efficacy evaluations for health applications could be expanded to include **architectural safety characteristics**—especially those that affect behavior under degraded stability—consistent with calls for standards that evaluate how digital health is integrated into health systems [22].
- **Provisional Implication:** For applications indicated for chronic pain, mental health, or post-operative care, the inability to provide offline access or local sovereignty may represent a clinically relevant risk factor. Evidence on internet-delivered chronic pain interventions and persistent disparities in patient-facing portal use underscores that access and capability constraints can shape who benefits and who is excluded [13, 7, 30]. Control-based verification regimes and evaluation guidance in digital health can inform how such risks are surfaced and studied [33].

16.3 For Law: The Doctrine of “Digital Duress”

Contract law relies on the “Meeting of the Minds.” We argue that no such meeting occurs when one party is an algorithm and the other is a human in a Vulnerability State (S_2, S_3).

- **The Shift:** “Terms of Service” agreements or data waivers executed during a detected Crisis State (S_2) could be analyzed as **presumptively voidable** under doctrines that already address impaired consent (e.g., duress, unconscionability, incapacity).
- **Provisional Implication: Data Duress** is proposed as a legal-technical concept for contexts where interaction constraints and system design materially narrow choice. This paper does not assert a settled doctrinal outcome; it identifies a plausible line of argument and a set of technical conditions relevant to it.

This section is intentionally provisional. A fuller interdisciplinary treatment would examine (i) how protective controls and evidence artifacts interact with product liability and negligence claims, (ii) whether specific domains (health, finance, safety tools) merit affirmative requirements for protective properties, and (iii) how protective constraints intersect with emerging AI governance regimes (e.g., requirements about risk management, transparency, and monitoring).

17 Limitations and Evidence Requirements

This paper introduces a framework and feasibility-oriented analysis; it does not provide outcome evaluation of effectiveness in real-world deployments. The primary limitations and corresponding evidence requirements are as follows.

17.1 No Original User Studies (Yet)

This manuscript does not report original qualitative or quantitative user studies with populations operating in S_2/S_3 conditions. As a result, claims about user experience, acceptability, and real-world safety outcomes remain unvalidated. Evidence requires participatory research with appropriate safeguards, as outlined in Section *Human Grounding and Participatory Design Considerations*.

17.2 Feasibility Artifact, Not Efficacy Proof

The reference implementation (PainTracker) is used as an illustrative feasibility artifact showing that a protective architecture is technically realizable within contemporary constraints. It is not evidence of clinical efficacy, improved outcomes, or superior safety relative to alternatives. Any such claims would require study designs appropriate to the domain (e.g., mixed-methods evaluations, longitudinal deployment studies, and auditing of security-critical subsystems).

17.3 PLS Is a Proposed Metric (Unvalidated)

The Protective Legitimacy Score (PLS), tier thresholds, and accountability dimensions are proposed constructs intended to support comparative evaluation. They require evaluation work including (i) construct validity studies, (ii) inter-rater reliability for scoring and review processes, (iii) domain calibration of weights and thresholds, and (iv) evaluation of gaming resistance when metrics are used in organizational settings.

17.4 State Detection and Mode Switching Are Unresolved

Correctly detecting Vulnerability State (or triggering appropriate protective modes) is an “Achilles’ heel” for protective systems: false positives can constrain autonomy and access, while false negatives can fail to protect users when protection is needed. Any state detection mechanism must therefore be evaluated as a safety-critical component, with explicit measurement of error rates, user override behavior, and the consequences of misclassification. This limitation is elevated as a primary research frontier rather than treated as an implementation detail.

17.5 Scope and Threat Boundaries

Protective Computing is not a claim of comprehensive security. It does not address compromised operating systems, malware, or adversaries with physical control of devices, and it cannot eliminate risk under coercion. The framework focuses on architectural properties that reduce harm and preserve agency under degraded stability, but must be complemented by domain-specific threat modeling and governance.

18 Research Agenda & Grand Challenges (2026–2030)

The Overton Framework establishes the axioms, but maturing Protective Computing requires a staged evidence program. This section outlines an evaluation roadmap (methods, populations, safeguards, and evaluation criteria) and identifies technical barriers that require research contributions.

18.1 Evaluation Roadmap (Staged Program)

We propose a staged program aligned with CHI expectations for theory + design research contributions:

1. **Stage 1 — Participatory grounding (0–12 months):** Partner with community organizations and advisory participants to refine failure scenarios, acceptable tradeoffs, and “safe exit” requirements. Methods: co-design workshops, scenario walkthroughs, and structured interviews using participant-controlled safety protocols.
2. **Stage 2 — Controlled feasibility evaluation (6–18 months):** Evaluate prototypes under constrained conditions that approximate degraded stability without inducing harm. Methods: lab-based or remote structured tasks with simulated connectivity/power constraints, comprehension checks, and accessibility-focused evaluation.

3. **Stage 3 — Field deployment and audit (12–36 months):** Conduct longitudinal pilots in domain-appropriate settings with independent audits of security-critical subsystems and measurement of incident trajectories (e.g., lockout events, data loss, safe-exit success). Methods: mixed-methods field studies, audit reports, and post-incident interviews.

Across stages, evaluation criteria SHOULD prioritize: (i) safe exit success under stress, (ii) reversibility of destructive actions, (iii) continuity of essential utility under instability, (iv) user-perceived control and dignity, and (v) minimization of additional surveillance risk.

18.2 Challenge I: The “Lost Key” Paradox (Cryptographic Usability)

Problem: Protective Computing mandates Client-Side Encryption (Principle 2). However, users in trauma states (S_2) frequently experience memory deficits. Loss of the key results in catastrophic data loss (Type I Harm).

Research Target: Development of **recovery mechanisms** (e.g., social recovery protocols such as Shamir’s Secret Sharing variants) that enable restoration without exposing plaintext to centralized services. Evaluation criteria SHOULD include recovery success under realistic user constraints, coercion resistance properties, and clear failure behavior when recovery is not possible.

18.3 Challenge II: State Detection and Mode Switching without Surveillance

Problem: Many protective behaviors depend on detecting vulnerability state or switching modes appropriately (e.g., enabling simplified interfaces, suppressing non-critical prompts, or triggering panic routines). Incorrect detection can either constrain autonomy (false positives) or fail to protect (false negatives). Telemetry-heavy approaches risk introducing additional surveillance.

Research Target: Develop **local-only** and **user-controllable** state detection approaches and evaluate them as safety-critical components. Evaluation criteria SHOULD include misclassification rates, usability of override/exit mechanisms, accessibility impacts, and the downstream harm of both false positives and false negatives.

18.4 Challenge III: Offline AI for Vulnerable Contexts

Problem: Current AI (LLMs) relies on massive cloud connectivity. A user in a shelter or disaster zone cannot access legal or medical synthesis without exposing their query to

corporate surveillance.

Research Target: Optimization of **Small Language Models (SLMs)** capable of running on consumer hardware to provide legal/medical synthesis offline. Evaluation criteria **SHOULD** include privacy characteristics (no required network calls), reliability under intermittent power/connectivity, and calibrated presentation of uncertainty for safety-adjacent domains.

19 Conclusion: From Convenience to Sanctuary

The history of consumer computing has prioritized **convenience**: reducing the effort required to transact, communicate, and consume. In high-vulnerability contexts, however, convenience-optimized assumptions can conflict with safety, agency, and continuity of access.

The **Overton Framework** argues for a shift from “stability-first” design toward systems that explicitly preserve user agency and essential utility under degraded conditions.

In domains where software meaningfully shapes outcomes for health, safety, or rights, architectural choices function as infrastructure choices. A central claim of Protective Computing is therefore pragmatic: systems should remain usable, contain failure, and support safe exit when the user cannot reliably supply stability.

Colophon

This document presents a scope-locked **Canon** for the Overton Framework for Protective Computing.

Through successive analytical and editorial refinement, it establishes:

- **Academic legitimacy** through formal models, threat specification, and evidence grounding
- **Engineering applicability** through RFC-style normative principles and reference architecture
- **Governance relevance** through accountability mechanisms and discussion of legal and policy implications (provisional)
- **Disciplinary scope** through positioning within prior work and articulation of grand challenges

The framework is therefore positioned for **submission, citation, evaluation, and community refinement**.

Its further evaluation now depends not on authorship, but on **independent implementation, review, and use**.

References

- [1] Amazon Web Services. Summary of the Amazon S3 service disruption in the northern virginia (US-EAST-1) region. Post-incident summary, 2017.
- [2] Association for Progressive Communications (APC). Feminist principles of the internet (version 2). Web resource, 2016.
- [3] Algirdas Avizienis, Jean-Claude Laprie, Brian Randell, and Carl Landwehr. Basic concepts and taxonomy of dependable and secure computing. *IEEE Transactions on Dependable and Secure Computing*, 1(1):11–33, 2004.
- [4] Lisanne Bainbridge. Ironies of automation. *Automatica*, 19(6):775–779, 1983.
- [5] Shaowen Bardzell. Feminist hci. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 1301–1310. ACM, 2010.
- [6] Scott Bradner. Key words for use in RFCs to indicate requirement levels. RFC 2119, March 1997.
- [7] Monica Buhrman, Torsten Gordh, and Gerhard Andersson. Internet interventions for chronic pain including headache: A systematic review. *Internet Interventions*, 4:7–34, 2016.
- [8] Ann Cavoukian. Privacy by design: The 7 foundational principles. Information and Privacy Commissioner of Ontario, 2009.
- [9] James I. Charlton. *Nothing About Us Without Us: Disability Oppression and Empowerment*. University of California Press, 1998.
- [10] Coalition Against Stalkerware. Stalkerware detection, removal and prevention (information for survivors). Web resource, n.d. No publication date listed; accessed 2026-02-17.
- [11] Sasha Costanza-Chock. *Design Justice: Community-Led Practices to Build the Worlds We Need*. The MIT Press, 2020.
- [12] Morris J. Dworkin. Recommendation for block cipher modes of operation: Methods and techniques. Technical report, National Institute of Standards and Technology (NIST), 2001.
- [13] Christopher Eccleston, Emma Fisher, Lorraine Craig, Geoffrey B. Duggan, Benjamin A. Rosser, and Edmund Keogh. Psychological therapies (internet-delivered) for the management of chronic pain in adults. *Cochrane Database of Systematic Reviews*, 2014.

- [14] Alex Feyerke. Offline first. Blog post, 2013.
- [15] Diana Freed, Jack Palmer, Diego Minchala, Karen Levy, Thomas Ristenpart, and Nicola Dell. A stalker’s paradise: How intimate partner abusers exploit technology. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. ACM, 2018.
- [16] Batya Friedman, Jr. Kahn, Peter H., and Alan Borning. Value sensitive design: Theory and methods. *Communications of the ACM*, 45(1):17–23, 2002.
- [17] Erik Hollnagel, David D. Woods, and Nancy Leveson, editors. *Resilience Engineering: Concepts and Precepts*. Ashgate, 2006.
- [18] Institute of Medicine. *Health IT and Patient Safety: Building Safer Systems for Better Care*. The National Academies Press, 2012.
- [19] Intergovernmental Panel on Climate Change. Climate change 2023: Synthesis report. summary for policymakers. Technical report, IPCC, 2023.
- [20] Lilly Irani, Janet Vertesi, Paul Dourish, Kavita Philip, and Rebecca E. Grinter. Postcolonial computing. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 1311–1320. ACM, 2010.
- [21] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. Local-first software: you own your data, in spite of the cloud. Ink & Switch (essay), 2019.
- [22] Alain Labrique, Lavanya Vasudevan, William Weiss, and Kate Wilson. Establishing standards to evaluate the impact of integrating digital health into health systems. *Global Health: Science and Practice*, 6(Supplement 1):S5–S17, 2018.
- [23] Barry Leiba. Ambiguity of uppercase vs lowercase in RFC 2119 key words. RFC 8174, May 2017.
- [24] Nancy G. Leveson. *Safeware: System Safety and Computers*. Addison-Wesley, 1995.
- [25] National Institute of Standards and Technology (US). Secure hash standard. Technical report, National Institute of Standards and Technology (NIST), 2015.
- [26] National Institute of Standards and Technology (US). Advanced encryption standard (aes). Technical report, National Institute of Standards and Technology (NIST), May 2023.

- [27] National Network to End Domestic Violence (NNEDV), Safety Net Project. The safety net survivor’s guide to ai: Understanding artificial intelligence, protecting your autonomy, and making empowered choices. Web resource, n.d. No publication date listed; accessed 2026-02-17.
- [28] Jens Rasmussen. Skills, rules, and knowledge; signals, signs, and symbols, and other distinctions in human performance models. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-13(3):257–266, 1983.
- [29] James Reason. *Human Error*. Cambridge University Press, 1990.
- [30] U. Sarkar, A. J. Karter, J. Y. Liu, N. E. Adler, R. Nguyen, A. Lopez, and D. Schillinger. Social disparities in internet patient portal use in diabetes: evidence that the digital divide extends beyond access. *Journal of the American Medical Informatics Association*, 18(3):318–321, 2011.
- [31] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*, 2011.
- [32] Jacob O. Wobbrock, Shaun K. Kane, Krzysztof Z. Gajos, Susumu Harada, and Jon Froehlich. Ability-based design. *ACM Transactions on Accessible Computing*, 3(3), 2011.
- [33] World Health Organization. Who guideline: Recommendations on digital interventions for health system strengthening. Technical report, World Health Organization, 2019.
- [34] World Health Organization. Global strategy on digital health 2020–2025. Policy document, 2020.